

DS 4400: Machine Learning and Data Mining I

Spring 2026 — Final Project Report

Project Title: Credit Card Fraud Detection: A Comparative Machine Learning Study on the IEEE-CIS Dataset
Team Members: Bridget Crampton
Code: <https://colab.research.google.com/drive/1mttqg-Und0nJOL8to6hhKmJziD->

Problem Description

Problem Statement

Given a set of transaction and identity features, predict whether a transaction is fraudulent or legitimate. This is a binary classification problem using the IEEE-CIS Fraud Detection dataset, evaluated on AUC-ROC.

Motivation

Credit card fraud results in billions of dollars in losses annually for consumers and financial institutions. Payment card fraud losses worldwide reached \$33.83 billion in 2023, with the United States accounting for over 42% of global losses (The Nilson Report, 2025). Automated fraud detection systems that can flag suspicious transactions in real time are critical for minimizing these losses (Cherif et al., 2022). This dataset presents several challenges that make it a realistic and interesting ML problem: severe class imbalance (~3.5% fraud), high dimensionality (400+ features), anonymized features with no documented meaning, and temporal structure that requires careful handling during model evaluation.

Related Work

- Chung, J. and Lee, K. “Credit Card Fraud Detection: An Improved Strategy for High Recall Using KNN, LDA, and Linear Regression.” *Sensors*, 2023. This paper proposes an ensemble optimized for recall on the IEEE-CIS dataset, achieving a recall of 0.9362. It reinforces the importance of prioritizing recall over accuracy in fraud detection, since missing a fraudulent transaction carries a higher cost than a false alarm. <https://doi.org/10.3390/s23187788>
- Xiao, Z. “IEEE-CIS Fraud Detection Based on XGB.” *Proceedings of the 7th International Conference on Economic Management and Green Development (ICEMGD)*, Springer, 2024. This paper applies an ensemble of CatBoost, XGBoost, and LightGBM directly to the IEEE-CIS dataset with detailed preprocessing and feature engineering pipelines, providing a direct benchmark for our XGBoost model. https://doi.org/10.1007/978-981-97-0523-8_159
- Deotte, C. “Leveraging Machine Learning to Detect Fraud: Tips to Developing a Winning Kaggle Solution.” *NVIDIA Technical Blog*, 2022. This post describes the first-place Kaggle IEEE-CIS solution, achieving 0.9459 AUC with an ensemble of XGBoost, CatBoost, and LightGBM. It provides a practical performance ceiling for this dataset and motivated our feature engineering approach. <https://developer.nvidia.com/blog/leveraging-machine-learning-to-detect-fraud-tips-to-developer>
- Homaei, H. et al. “Robust Fraud Detection with Ensemble Learning: A Case Study on the IEEE-CIS Dataset.” *Preprints.org*, 2025. This paper benchmarks Random Forest, XGBoost, LightGBM, and stacking ensembles on the IEEE-CIS dataset, achieving 0.918 AUC-ROC with a stacking approach, and evaluates SMOTE and ADASYN for class imbalance. <https://www.preprints.org/manuscript/202507.1711>

Dataset

Dataset Statistics

- **Source:** IEEE-CIS Fraud Detection dataset from Kaggle (<https://www.kaggle.com/competitions/ieee-fraud-detection>), provided by Vesta Corporation.
- **Records:** 590,540 transactions in the training set across two tables (transaction and identity) joined on `TransactionID`. Only 144,233 transactions (24.4%) have a corresponding identity record.
- **Features:** 434 total features after merging, broken down as: numerical (`TransactionAmt`, `C1–C14`, `D1–D15`, `V1–V339`), categorical (`ProductCD`, `card4`, `card6`, email domains, identity columns `id_12–id_38`), and device features (`DeviceType`, `DeviceInfo`).
- **Target variable:** `isFraud` is severely imbalanced: 96.5% legitimate and 3.5% fraudulent (20,663 fraud cases).
- **Missing data:** Only 21 of 434 columns are fully complete. 74 columns exceed 80% missingness. The `V` columns are structurally sparse, populated only for certain `ProductCD` values. All identity columns are null for transactions with no identity record (75.6% of the training set).

Feature Definitions and Insights

The dataset contains features across six groups. Key features are defined below.

Feature	Type	Description
<code>TransactionAmt</code>	Numerical	Transaction amount in USD. Right-skewed; nearly identical distribution between fraud and legitimate transactions, suggesting amount alone is a weak fraud signal.
<code>ProductCD</code>	Categorical	Product category (W, H, C, S, R). Determines which <code>V</code> feature blocks are populated, making missingness structural.
<code>card1--card6</code>	Mixed	Card identifiers, network (visa/mastercard/discover), and category (debit/credit).
<code>C1--C14</code>	Numerical	Anonymized count features. <code>C1</code> , <code>C2</code> , and <code>C4</code> are nearly perfectly correlated ($r > 0.97$).
<code>D1--D15</code>	Numerical	Time deltas in days since prior events. Null values filled with <code>-1</code> as a sentinel.
<code>M1--M9</code>	Boolean	Match flags for card, name, and address. NaN treated as a third category.
<code>V257</code> , <code>V201</code> , <code>V200</code>	Numerical	Highest correlated Vesta features with <code>isFraud</code> ($r \approx 0.25$); inter-feature correlations of 0.70–0.98.
<code>DeviceType</code>	Categorical	Mobile vs. desktop.
<code>has_identity</code>	Boolean	Engineered flag: 1 if a transaction has a corresponding identity record.

Table 1: Key features in the IEEE-CIS Fraud Detection dataset.

Exploratory Data Analysis

Figure 1 shows the severe class imbalance in the dataset, with fraud comprising only 3.5% of all transactions. This imbalance motivates the use of `class_weight='balanced'` across all models and AUC-ROC as

the primary evaluation metric rather than accuracy.

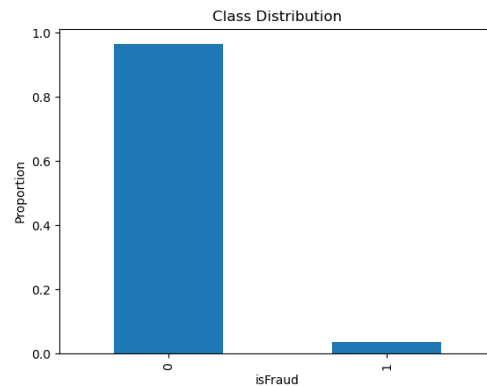


Figure 1: Class distribution showing severe imbalance: 96.5% legitimate transactions vs. 3.5% fraudulent.

Figure 2 shows the transaction amount distributions for fraud and legitimate transactions. Both are heavily right-skewed and nearly identical in shape, indicating that transaction amount alone is not a strong discriminator between fraud and legitimate transactions.

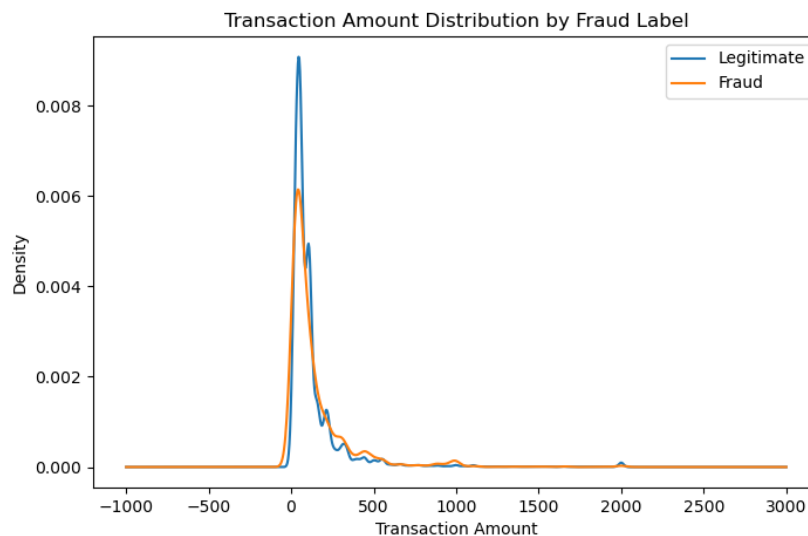


Figure 2: KDE plot of transaction amounts (clipped at \$2,000) by fraud label. Both distributions are nearly identical, confirming that transaction amount alone has weak predictive power for fraud.

Figure 3 shows the correlation heatmap of the 15 features most correlated with `isFraud`. All top features are `V` columns with correlations of 0.20–0.25. The high inter-feature correlations (0.70–0.98) among `V` features present a multicollinearity challenge for linear models but do not affect tree-based models.

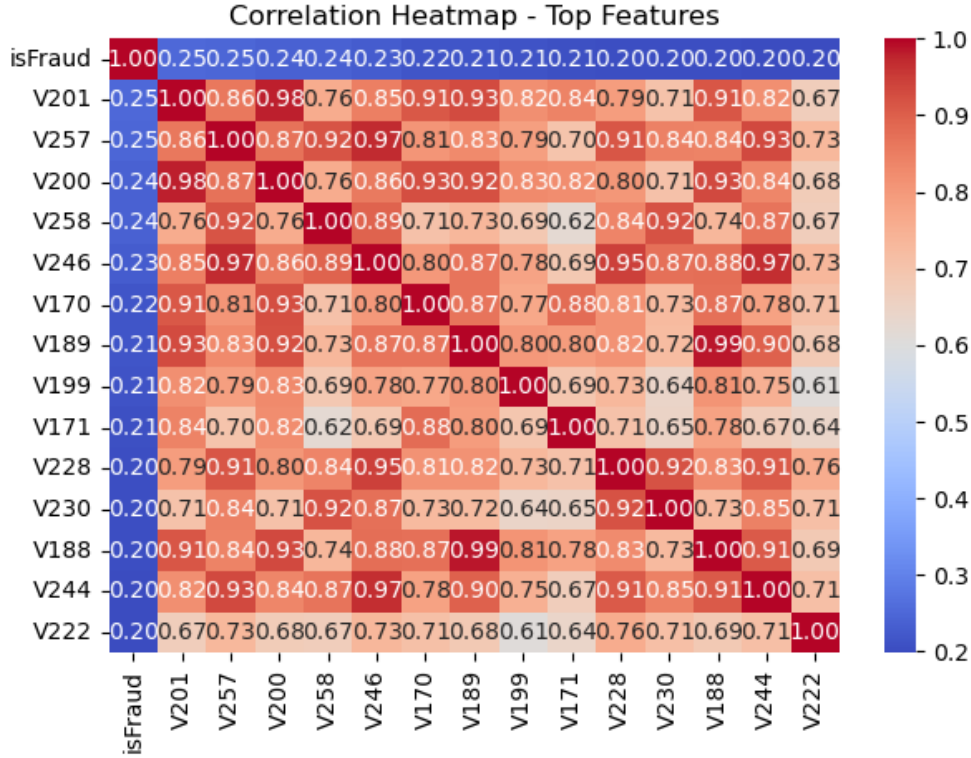


Figure 3: Correlation heatmap of the 15 features most correlated with `isFraud`. Top features are all V columns with $r \approx 0.25$. High inter-feature correlations suggest multicollinearity that limits linear model performance.

Approach and Methodology

General Approach

Our pipeline follows supervised binary classification. We merge the transaction and identity tables, engineer new features, clean and impute missing values, encode categorical features, and train six models of increasing complexity: logistic regression as a linear baseline, random forest, XGBoost, a standard neural network with batch normalization, a custom fraud-specific neural network, and a stacking ensemble. All models are evaluated on AUC-ROC using a time-ordered 80/20 train/validation split (`shuffle=False`) to prevent future data leakage, as the dataset contains transactions ordered chronologically by `TransactionDT`.

Preprocessing and Feature Engineering

Feature engineering. New features were created before imputation to preserve raw value integrity. Time-based features were extracted from `TransactionDT`: hour of day, day of week, `is_night` (hours 0–6), and `is_weekend`. Card-based aggregations were computed from the training set: transaction count per card (`card1_count`), mean and standard deviation of transaction amount per card, and amount deviation from the card mean (`amt_deviation`), capturing how unusual a given transaction is for that card. Email features include `email_match` (whether purchaser and recipient email domains match) and `common_email` (whether the domain is gmail, yahoo, hotmail, or outlook). `is_round_amt`, a binary flag for whether the transaction amount is a whole number, as round amounts have been observed as a behavioral pattern in fraudulent transactions in prior Kaggle competition analysis (Deotte, 2022). Address features include transaction count per address (`addr1_count`) and fraud rate per address (`addr1_fraud_rate`).

using target encoding computed from training statistics only.

Missing value imputation. Missing values were handled differently depending on feature type, following the principle that the imputation strategy should reflect the semantic meaning of missingness for each feature group rather than applying a uniform approach (Garcia et al., 2010).

Count features (C1–C14) were filled with 0, as a missing count indicates no observed activity for that entity rather than a truly unknown value. Time delta features (D1–D15) were filled with -1 as a sentinel value outside the natural range of the feature, allowing tree-based models to learn that -1 represents no prior event on record rather than a small elapsed time. Using the median would blend these rows with legitimate records that simply had short time deltas, obscuring the distinction.

Match flags (M1–M9) were encoded as 1 (True), 0 (False), and 2 (missing), treating NaN as a meaningful third category rather than imputing T or F. The absence of a match flag likely indicates the verification was not performed, which is itself informative and distinct from a verified match or mismatch.

Numeric identity columns (id_01 through id_11) were filled with -1 as a sentinel value to distinguish transactions with no identity record from those with legitimately low identity scores. Filling with the median would incorrectly group no-identity transactions with low-risk legitimate ones. Categorical identity columns were filled with `Unknown` to preserve the category structure while flagging the absence of an identity record.

V columns were filled with -1 as a sentinel, since their missingness is structural. V features are only populated for certain values of `ProductCD`, meaning absence is informative rather than random. Columns exceeding 80% missingness (74 columns) were retained but imputed with -1 , following the principle that the structural nature of this missingness carries predictive signal and should be preserved rather than discarded (Salgado et al., 2016).

Binary missing indicator flags were added for `dist1`, `dist2`, and the sparsest D columns (D2–D9, D15), following the recommendation of Van Buuren (2018) that missingness itself can carry predictive signal and should be explicitly encoded when the mechanism of missingness is informative rather than completely at random.

Encoding. High cardinality categorical columns (`addr1`, `addr2`, email domains, `DeviceInfo`, select identity columns) were frequency encoded using training set statistics only. Low cardinality columns (`ProductCD`, `card4`, `card6`, `DeviceType`, select identity flags) were label encoded.

Scaling. Features were standardized using `StandardScaler` fitted on the training split only. Scaled features are used for logistic regression and neural networks; tree-based models use unscaled features.

Train/validation split. The training data was split 80/20 using `shuffle=False` to preserve temporal order, producing 472,432 training samples and 118,108 validation samples. Shuffling would leak future transaction patterns into the training set, as transactions are ordered chronologically by `TransactionDT` and a shuffled split would allow the model to train on transactions that occurred after those in the validation set.

Models Trained

Logistic Regression

- *Configuration:* `class_weight='balanced'`, `max_iter=1000`, `C=0.1` (best from sweep over $\{0.01, 0.1, 1.0, 10.0\}$), `random_state=42`.
- *Train AUC:* 0.8802 *Val AUC:* 0.8316 *Gap:* 0.0486
- *Observations:* Logistic regression serves as a reasonable linear baseline but suffers from low fraud

class precision (0.10) due to severe class imbalance and the non-linear nature of the fraud signal. High multicollinearity among V features ($r = 0.70\text{--}0.98$) also limits performance. AUC plateaued around 0.832 regardless of regularization strength, indicating the model has hit its ceiling on this dataset.

Random Forest

- *Configuration:* `n_estimators=100, max_depth=10, class_weight='balanced', random_state=42`. Selected from sweep over estimators {10, 50, 100, 500} and depths {5, 10, 15, 20, None}.
- *Train AUC:* 0.9079 *Val AUC:* 0.8688 *Gap:* 0.0391
- *Observations:* Random forest substantially outperformed logistic regression across all metrics. `max_depth=10` provided the best bias-variance tradeoff. Unlimited depth achieved higher val AUC (0.8918) but with a larger gap (0.108), while `depth=5` underfitted (val AUC 0.8423). The small gap of 0.0391 indicates good generalization.

XGBoost

- *Configuration:* `n_estimators=500, learning_rate=0.05, max_depth=6, subsample=0.8, colsample_bytree=0.8, scale_pos_weight≈27, random_state=42`.
- *Train AUC:* 0.9766 *Val AUC:* 0.9075 *Gap:* 0.0691
- *Observations:* XGBoost is the strongest individual model, outperforming all others across all metrics. Adding `subsample=0.8` and `colsample_bytree=0.8` improved val AUC from 0.9006 to 0.9063 before feature engineering, and to 0.9075 after. The feature engineering additions (amount deviation, time features, email match) contributed a 0.0012 AUC improvement.

Neural Network

Three neural network variants were trained iteratively. `nn_1` (`lr=0.001`, `Dropout=0.3`) served as the baseline and exhibited unstable training with validation loss spikes. `nn_2` (`lr=0.0001`, `Dropout=0.4`) reduced instability by lowering the learning rate but did not fully resolve the spikes. `nn_3` added batch normalization on top of `nn_2`'s configuration, which fully stabilized training. The results below reflect the best performing variant (`nn_3`) unless otherwise noted.

- *Configuration (nn_1):* 3-layer feedforward network (256, 128, 64), ReLU activation, `Dropout(0.3)`, `Adam lr=0.001`, 30 epochs.
- *Configuration (nn_3, with batch normalization):* Same architecture with `BatchNormalization` after each hidden layer, `Dropout(0.4)`, `Adam lr=0.0001`, 30 epochs.
- *Train AUC:* 0.9253 *Val AUC:* 0.8687 *Gap:* 0.0566
- *Observations:* The standard neural network (`nn_1`) showed unstable training with validation loss spikes caused by internal covariate shift. Adding batch normalization (`nn_3`) produced a smooth, stable loss curve but val AUC remained around 0.868 across all variants. Neural networks consistently underperformed XGBoost on this tabular dataset, consistent with existing literature showing gradient boosting dominates on structured data (Grinsztajn et al., 2022).

Metrics and Model Comparison

All models are evaluated using AUC-ROC as the primary metric, as it is the official Kaggle competition metric and is robust to class imbalance. Unlike accuracy, AUC-ROC evaluates model performance across all classification thresholds and is unaffected by class distribution, making it well-suited for severely imbalanced datasets such as this one (Fawcett, 2006). Additional metrics are reported on the fraud class (label = 1) specifically, as overall accuracy is misleading given the 96.5/3.5 class split.

AUC-ROC measures the model’s ability to distinguish between fraud and legitimate transactions across all classification thresholds. A score of 1.0 is perfect and 0.5 is random.

Precision (fraud class) measures what proportion of flagged transactions are actually fraudulent. Low precision means many false alarms, creating unnecessary friction for legitimate customers.

Recall (fraud class) measures what proportion of actual fraud cases are caught. Low recall means missed fraud and direct financial losses. In fraud detection, recall is generally prioritized since the cost of missing fraud exceeds the cost of a false alarm.

F1-score (fraud class) is the harmonic mean of precision and recall, providing a single metric that balances both, particularly useful for imbalanced datasets.

Accuracy measures overall correct predictions. While high across all models due to class imbalance, it is not a reliable fraud detection metric. A model predicting all transactions as legitimate would achieve 96.5% accuracy while catching zero fraud.

Model	AUC	Accuracy	Precision	Recall	F1	Gap
Logistic Regression	0.8316	0.75	0.10	0.77	0.17	0.0480
Random Forest	0.8688	0.88	0.18	0.68	0.28	0.0383
XGBoost	0.9075	0.92	0.26	0.71	0.38	0.0644
Neural Network	0.8687	0.82	0.13	0.75	0.22	0.0891
Custom NN	0.8597	0.82	0.13	0.75	0.22	0.0665
Stacking Ensemble	0.8956	0.97	0.67	0.41	0.51	0.1023

Table 2: Model comparison across all evaluation metrics on the validation set. Precision, Recall, and F1 are reported on the fraud class (label = 1). Gap = Train AUC – Val AUC.

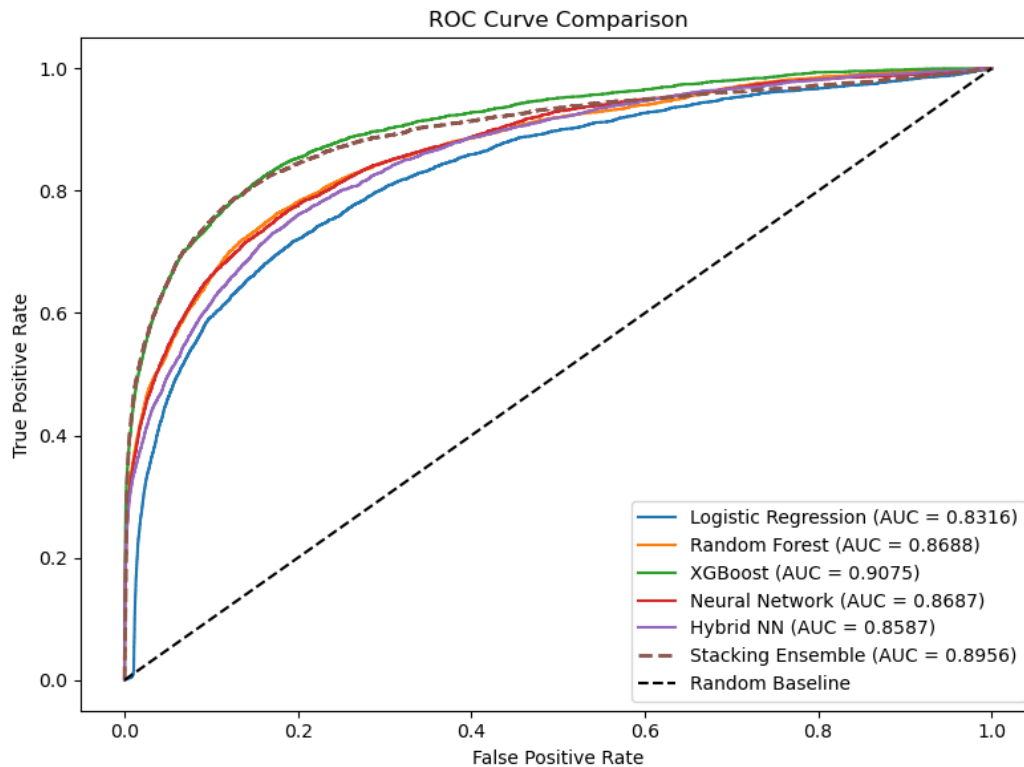


Figure 4: ROC curve comparison across all models. XGBoost achieves the highest AUC (0.9075), clearly separating from the other models. The custom NN (0.8597) and stacking ensemble (0.8956) follow, with random forest (0.8688), neural network (0.8687), and logistic regression (0.8316) completing the ranking. The dashed diagonal represents a random baseline (AUC = 0.5).

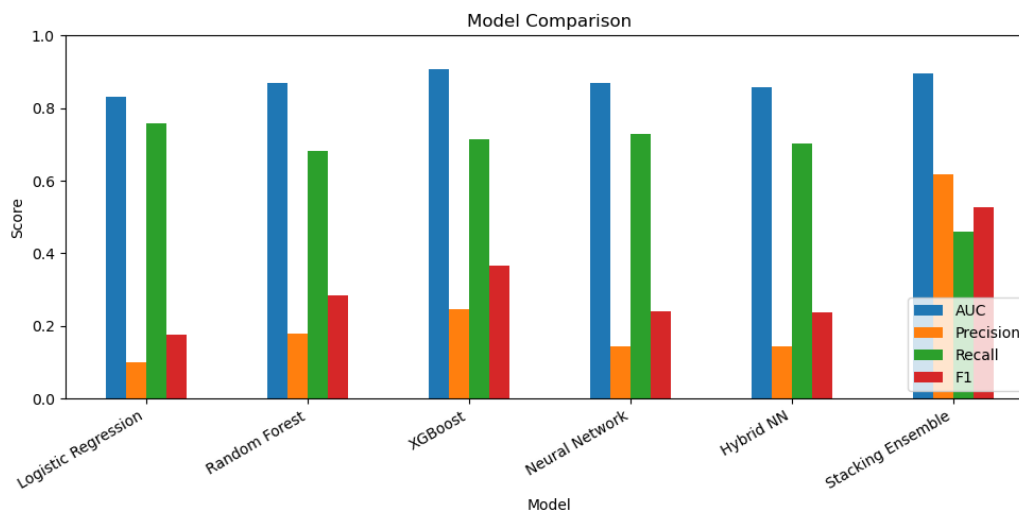


Figure 5: Bar chart comparing AUC, precision, recall, and F1 across all models on the fraud class.

Feature Importance

Figure 5 and Figure 6 show the top 20 most important features for Random Forest and XGBoost respectively. Both models agree on the importance of V features and C features (anonymized count features), with V70, V91, V294, C4, C8, and C14 appearing in both top 20 lists, confirming that these Vesta-engineered and count features carry the strongest fraud signal.

Notable differences exist in their rankings, which is expected given the different importance metrics used. Random Forest measures mean decrease in Gini impurity, distributing importance more evenly across features, while XGBoost measures gain, resulting in a more concentrated distribution where V258 alone accounts for 0.14 importance score, far exceeding all others. This concentration is typical of XGBoost’s gain metric, which rewards features that produce the largest improvement in loss per split.

The engineered feature `addr1_fraud_rate` appears in the Random Forest top 20, validating the target encoding feature engineering effort. The fraud rate per address carries meaningful predictive signal. Notably, `R_emaildomain` also appears in the Random Forest rankings, suggesting that the recipient email domain is informative for fraud detection. Neither `amt_deviation` nor `card1_count` appear in the top 20 for either model, suggesting their signal is present but weaker than the V and C features on this dataset.

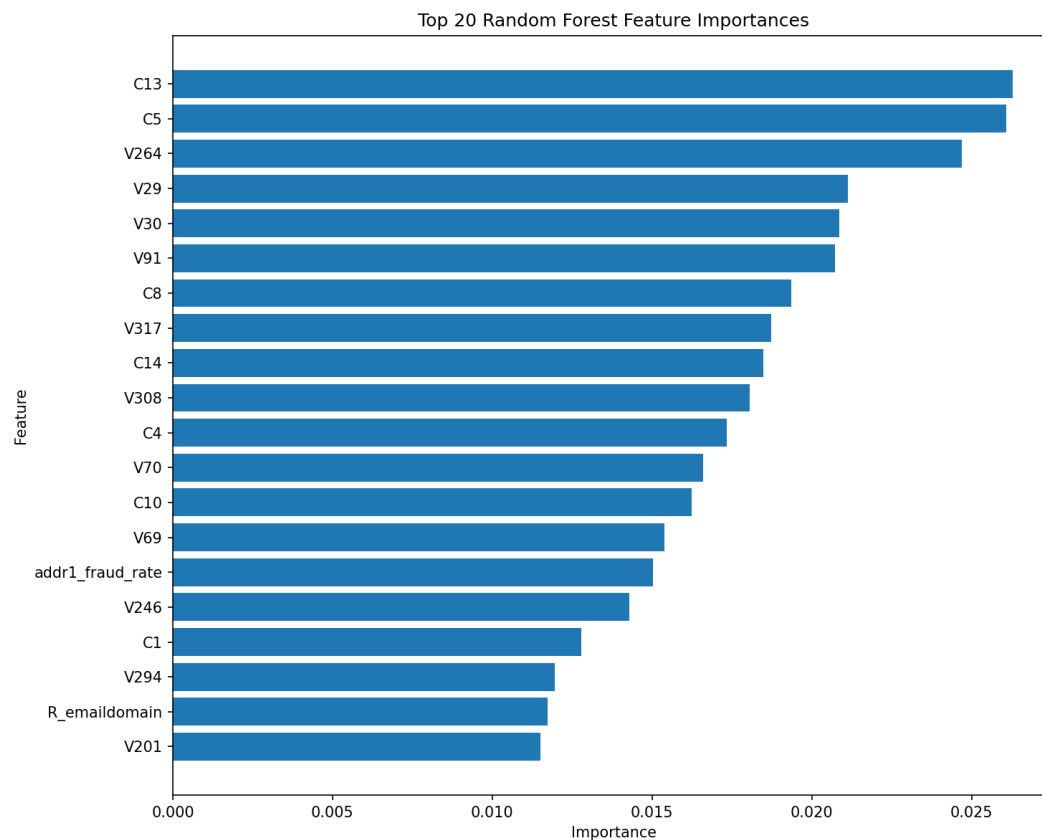


Figure 6: Top 20 Random Forest feature importances by mean decrease in Gini impurity. V features dominate, with several engineered features (`amt_deviation`, `card1_count`) also appearing in the top 20.

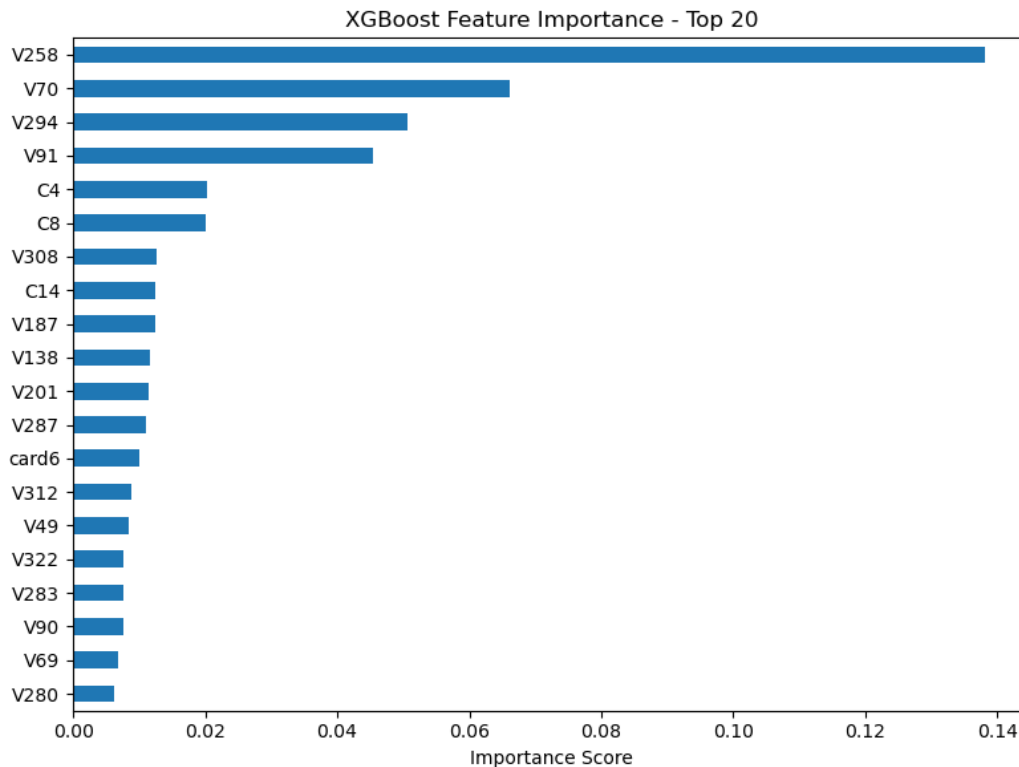


Figure 7: Top 20 XGBoost feature importances by gain. Agreement with Random Forest on the importance of V features and engineered features confirms their predictive value for fraud detection.

Neural Network Training Loss

Figures 8 and 9 show the training and validation loss curves for nn_1 (baseline) and nn_3 (with batch normalization). The comparison directly illustrates the effect of batch normalization on training stability.

nn_1 exhibits erratic validation loss spikes reaching as high as 0.35, caused by internal covariate shift. The distribution of inputs to each layer shifting as earlier layer weights update, making training unstable. nn_3 with batch normalization produces a smooth, consistent curve where both train and validation loss decrease steadily across all 30 epochs, with validation loss stabilizing around 0.105. Batch normalization addresses this by normalizing the outputs of each hidden layer to have zero mean and unit variance within each mini-batch, keeping activations in a stable range throughout training (<https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>).

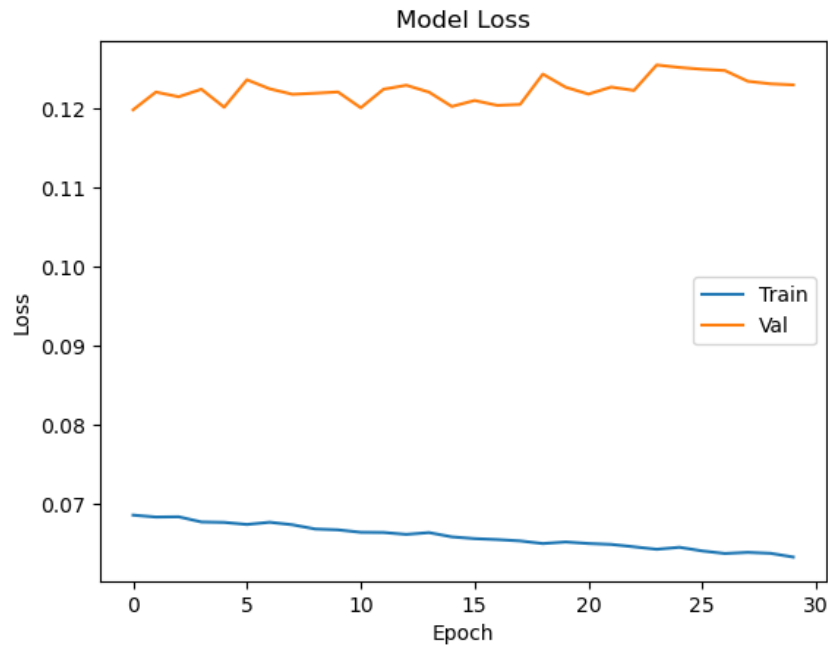


Figure 8: Training and validation loss for nn_1 (baseline, $\text{lr}=0.001$, $\text{Dropout}=0.3$). Erratic validation loss spikes indicate unstable training caused by internal covariate shift.

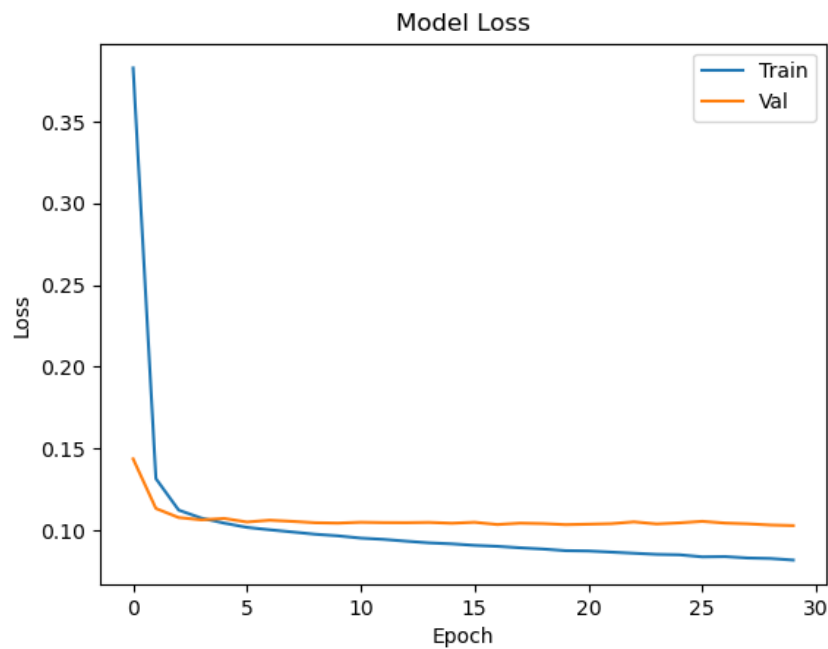


Figure 9: Training and validation loss for nn_3 (batch normalization, $\text{lr}=0.0001$, $\text{Dropout}=0.4$). Smooth, stable curves confirm that batch normalization effectively addresses training instability.

Insights and Discussion

Why logistic regression underperforms. The fraud signal in this dataset is non-linear. The V features that most correlate with fraud ($r \approx 0.25$) are all highly correlated with each other (0.70–0.98), creating severe multicollinearity that degrades linear model performance. Logistic regression’s val AUC plateaued at 0.832 regardless of regularization, indicating the model had reached its ceiling.

Why XGBoost dominates. Gradient boosting builds trees sequentially, each correcting the errors of the previous, allowing it to capture complex non-linear interactions between features. It also handles the class imbalance directly via `scale_pos_weight` and is largely immune to the multicollinearity that hurts logistic regression. This pattern is consistent with existing literature showing XGBoost outperforms other approaches on tabular fraud detection datasets.

Why neural networks underperform XGBoost. Neural networks struggled on this tabular dataset despite architecture tuning. This aligns with a well-established finding in the literature that tree-based models tend to outperform neural networks on structured tabular data, particularly when features are heterogeneous in scale and type as they are here (Grinsztajn et al., 2022).

Precision/recall tradeoff. All models using `class_weight= 'balanced'` show high recall but low precision on the fraud class. This is an intentional design choice: in fraud detection, missing actual fraud (false negative) is far more costly than flagging a legitimate transaction (false positive). The stacking ensemble is the notable exception, achieving high precision (0.67) at the cost of low recall (0.41). This is appropriate for contexts where false alarms are extremely costly.

Impact of feature engineering. Adding time-based features, card aggregations, email match flags, and amount deviation features improved XGBoost val AUC from 0.9063 to 0.9075 (a difference of 0.0012). The amount deviation feature in particular provides signal about whether a transaction is unusual for that card, which is a strong fraud indicator.

Novel Contributions

Custom Loss Function

A custom weighted binary crossentropy loss function was implemented to directly encode the asymmetric cost of fraud detection into the model’s objective. The standard binary crossentropy treats false positives and false negatives equally, but in fraud detection missing actual fraud is far more costly than a false alarm.

The motivation for a custom loss came from observing that even with `class_weight= 'balanced'` and dropout regularization, the standard neural network variants (`nn_1`, `nn_2`, `nn_3`) still produced low fraud class recall relative to XGBoost. Rather than adjusting the decision threshold after training, a custom loss function directly penalizes the model during training for assigning low probabilities to true fraud cases, making recall improvement part of the optimization objective itself.

The custom loss extends binary crossentropy by adding an additional penalty term that activates only when the true label is fraud ($y = 1$):

$$\mathcal{L}(y, \hat{y}) = \text{BCE}(y, \hat{y}) - \lambda \cdot y \cdot \log(\hat{y} + \epsilon)$$

where $\lambda = 3.0$ controls the penalty weight for false negatives, y is the true label, $\hat{y}$ is the predicted probability, and $\epsilon = 10^{-7}$ ensures numerical stability by preventing $\log(0)$. When $y = 0$ (legitimate transaction), the penalty term vanishes entirely, leaving standard binary crossentropy. When $y = 1$ (fraud), the additional $-\lambda \cdot \log(\hat{y})$ term further penalizes the model for assigning a low fraud probability, pushing it to assign

higher probabilities to fraud cases. The value $\lambda = 3.0$ was selected empirically. Values of 2.0, 3.0, 5.0, and 8.0 were tested, with 3.0 providing the best balance between improved recall and maintained AUC. This approach is directly motivated by cost-sensitive learning theory, which argues that misclassification costs should be incorporated into the loss function rather than handled post-hoc (Elkan, 2001).

This resulted in a val AUC of 0.8597 with a fraud recall of 0.75, consistent with the standard neural network (0.8687 AUC, 0.75 recall). While the custom loss did not produce a higher AUC, it encodes the asymmetric cost of fraud directly into the optimization objective, making the model's priorities explicit rather than relying solely on post-hoc class weighting. Class weights were additionally applied via `compute_class_weight('balanced')` to further address the 96.5/3.5 class split.

Hybrid Model: XGBoost Feature Selection and Neural Network

A hybrid architecture was developed combining XGBoost's feature selection capability with a neural network classifier. XGBoost feature importances were used to identify the top 50 most discriminative features, which were then used as the sole input to a neural network with batch normalization. The motivation is that XGBoost acts as an intelligent feature selection layer, providing the neural network with a focused, high-signal subset of the 400+ features rather than the full noisy feature space.

The hybrid neural network architecture consists of two hidden layers (128, 64 neurons) with batch normalization and Dropout(0.4) after each, trained with Adam at `lr=0.0001` for 30 epochs on the top 50 XGBoost features. This approach demonstrates how two models can be combined to leverage the strengths of each: XGBoost's ability to rank feature importance on tabular data, and the neural network's ability to learn non-linear transformations of those features.

Stacking Ensemble

A stacking ensemble was implemented combining four base models (logistic regression, random forest, XGBoost, and neural network) with logistic regression as the meta-learner. Rather than using raw features, the meta-learner trains on the predicted probabilities of the base models, learning patterns in how they agree and disagree (Geeksforgeeks, <https://www.geeksforgeeks.org/machine-learning/stacking-in-machine-learning/>).

The stacking ensemble achieved a validation AUC of 0.8956, outperforming logistic regression (0.8316) and random forest (0.8688), and showed a notable tradeoff compared to XGBoost: much higher fraud class precision (0.67 vs 0.26) at the cost of lower recall (0.41 vs 0.71). This makes the ensemble well-suited for contexts where false alarms are extremely costly.

A known limitation of this implementation is that base model training predictions were used to train the meta-learner rather than out-of-fold predictions, causing the meta-learner to train on overconfident in-sample predictions and producing a large overfitting gap (0.1023). The correct approach is out-of-fold stacking, where each base model predicts on held-out folds it never trained on, producing honest training estimates for the meta-learner (Geeksforgeeks, <https://www.geeksforgeeks.org/machine-learning/what-is-the-oof-prediction/>). Implementing proper OOF stacking is identified as a key area for future work.

Kaggle Submission

The final XGBoost model was submitted to the Kaggle IEEE-CIS Fraud Detection competition. Predictions were generated as fraud probabilities on the held-out test set and submitted in the required format of `TransactionID` and `isFraud` probability pairs.

Metric	Score
Public Leaderboard AUC	0.9195
Private Leaderboard AUC	0.8808
Validation AUC	0.9075

Table 3: Kaggle leaderboard scores vs. validation AUC for the final XGBoost model.

The public score (0.9195) is computed on a random 20% of the test set and is used for the live leaderboard. The private score (0.8808) is computed on the remaining 80% and is the more reliable estimate of true generalization performance. The small drop between validation AUC (0.9075) and private leaderboard AUC (0.8808) is expected given the temporal nature of the data. The test set covers transactions from a later time period than the training data, introducing mild distribution shift. For context, the winning solution in this competition achieved a private AUC of 0.9459 using an extensive ensemble of XGBoost, CatBoost, and LightGBM with deep feature engineering.

Team Member Contributions

- **Bridget Crampton:** Completed all work including data loading and merging, feature engineering, missing value imputation, categorical encoding, exploratory data analysis, train/validation splitting, model training and evaluation (logistic regression, random forest, XGBoost, neural network variants), novel contributions (custom loss function, hybrid model, stacking ensemble), and report writing.

References

- [1] Chung, J. and Lee, K. (2023). Credit card fraud detection: An improved strategy for high recall using KNN, LDA, and linear regression. *Sensors*, 23(18), 7788. <https://doi.org/10.3390/s23187788>
- [2] Xiao, Z. (2024). IEEE-CIS fraud detection based on XGB. In Li, X., Yuan, C., Kent, J. (Eds.), *Proceedings of the 7th International Conference on Economic Management and Green Development (ICEMGD 2023)*. Springer. https://doi.org/10.1007/978-981-97-0523-8_159
- [3] Deotte, C. (2022). Leveraging machine learning to detect fraud: Tips to developing a winning Kaggle solution. *NVIDIA Technical Blog*. <https://developer.nvidia.com/blog/leveraging-machine-learning/>
- [4] Homaei, H. et al. (2025). Robust fraud detection with ensemble learning: A case study on the IEEE-CIS dataset. *Preprints.org*. <https://www.preprints.org/manuscript/202507.1711>
- [5] Geeksforgeeks. Stacking in machine learning. <https://www.geeksforgeeks.org/machine-learning/stacking-in-machine-learning/>
- [6] Geeksforgeeks. What is the OOF (out-of-fold) approach? <https://www.geeksforgeeks.org/machine-learning/what-is-the-oofout-of-fold-approach/>
- [7] Geeksforgeeks. What is batch normalization in deep learning? <https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>
- [8] The Nilson Report. (2025). Payment card fraud losses worldwide reached \$33.83 billion in 2023. <https://www.globenewswire.com/news-release/2025/01/06/3004931/0/en/Payment-Card-fraud-losses-worldwide-reached-33-83-billion-in-2023.html>

- [9] Cherif, A., Badhib, A., Ammar, H., Alshehri, S., Kalkatawi, M., and Imine, A. (2022). Credit card fraud detection in the era of disruptive technologies: A systematic literature review. *Journal of King Saud University – Computer and Information Sciences*, 35, 145–174. <https://doi.org/10.1016/j.jksuci.2022.11.008>
- [10] Garcia, S., Luengo, J., and Herrera, F. (2010). A survey of discretization techniques: Taxonomy and empirical analysis in supervised learning. *IEEE Transactions on Knowledge and Data Engineering*, 25(4), 734–750.
- [11] Salgado, C. M., Azevedo, C., Proenca, H., and Vieira, S. M. (2016). Missing data. In *Secondary Analysis of Electronic Health Records*. Springer, Cham.
- [12] Van Buuren, S. (2018). *Flexible Imputation of Missing Data*. CRC Press. <https://stefvanbuuren.name/fimd/>
- [13] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>
- [14] Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520. <https://doi.org/10.48550/arXiv.2207.08815>
- [15] Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978. <https://cseweb.ucsd.edu/~elkan/rescale.pdf>